SANJIVANI COLLEGE OF ENGINEERING, KOPARGAON



(An autonomous Institute affiliated to SPPU Pune)

Department of Electronics & Computer Engineering

Academic Year: 2025-26 Sem-II

Industry 4.0

Title: KrushiAI: AI-IoT Soil Testing and Crop Recommendation System

Submitted By

Ms. Sakshi Vijay Thakare UEC23F1064

Ms. Sakshi Ramdas Zurale UEC23F1069

Course Incharge – Prof. A. D. Sangale

	Sanjivani Rural Educational Society's **SANJIVANI COLLEGE OF ENGINEERING** **(An Autonomous Institution)** **Kopargaon – 423 603, Maharashtra.**	**ACAD-F-15 K**
Academic Year: 2025-26	**CIA PART A ACTIVITY**	**Revision: 00** **Dated:**
Department:	**Electronics and Computer Engineering**	**Date of Preparation: 07/10/26**
Course Code & Name:	**Industry 4.0**	**Year/Sem: B.Tech / 7th sem**

Project Title: AI-IoT Soil Testing and Crop Recommendation System

Submitted by:

Roll No.	PRN No.	Name of the Student	Signature
64	UEC23F1064	Ms. Sakshi Vijay Thakare	
69	UEC23F1069	Ms. Sakshi Ramdas Zurale	

Prof. A. D. Sangale
(Course Incharge)

Dr. D. B. Kshirsagar
(Head of Department)

ABSTRACT

Crop selection is one of the most consequential decisions in agricultural production, yet it is frequently made on the basis of experience, local custom, and limited information about soil nutrient status and prevailing weather. Laboratory soil testing is reliable but is usually periodic, and integrated tools that combine soil measurements, location, weather context, and data-driven recommendation remain scarce for the individual farmer.

This report presents KrushiAI, an AI-IoT driven agricultural decision-support platform conceived to support the journey from crop selection to market profit. The long-term vision covers crop selection, IoT-based monitoring, storage, market analysis, and profit-oriented decision-making. The present prototype implements Module 1, the AI-Based Crop Recommendation System.

The prototype accepts soil information through manual entry or a clearly labelled developer sensor simulator, and a sensor architecture is designed around an ESP32 controller, an NPK sensor interfaced through an RS485/MAX485 module, a DHT11/DHT22 temperature and humidity sensor, and a soil moisture sensor. Nitrogen, phosphorus, potassium, soil moisture, air temperature, and humidity are processed by a FastAPI-based backend and presented through a React and Vite web interface. Location is supplied as latitude and longitude coordinates, and weather context is retrieved from the Open-Meteo service. A Random Forest classifier, implemented with Scikit-learn, uses six soil and environmental features to produce a ranked list of candidate crops with model scores. A local agricultural knowledge context supports the explanation of the recommendation, and the system generates English and bilingual English–Marathi PDF soil testing and crop recommendation reports for each soil test.

The software pipeline, from farmer registration to PDF report generation, was exercised through the API and the web application. The Random Forest model, trained on a small demonstration dataset covering multiple crop classes, reached approximately 40% validation accuracy as recorded by the team. This figure is reported transparently: the dataset is intended to validate the software pipeline rather than to deliver production-grade performance, and the reported scores are model outputs rather than guarantees of crop success. The physical sensor path is a proposed architecture and has not been verified end to end. KrushiAI is therefore presented as a decision-support prototype whose recommendations should be validated with local agricultural experts and laboratory soil testing.

Planned future work includes a larger and regionally representative dataset, improved models, retrieval-augmented agricultural knowledge, real-time IoT monitoring, storage monitoring, market price analysis, a multilingual mobile interface, and cloud deployment.

TABLE OF CONTENTS

ABSTRACT ii
CHAPTER 1 — INTRODUCTION 1
CHAPTER 2 — EXISTING SYSTEM AND PROBLEM STATEMENT 3
CHAPTER 3 — PROPOSED SYSTEM 5
CHAPTER 4 — COMPONENTS AND TECHNOLOGIES USED 8
CHAPTER 5 — METHODOLOGY AND IMPLEMENTATION 10
CHAPTER 6 — RESULTS AND OUTPUT 13
CHAPTER 7 — ADVANTAGES AND LIMITATIONS 22
CHAPTER 8 — APPLICATIONS 23
CHAPTER 9 — FUTURE SCOPE 24
CONCLUSION 25
REFERENCES 26

LIST OF FIGURES

Figure 1.1: Overview of the KrushiAI platform..... 2
Figure 3.1: KrushiAI System Architecture 6
Figure 3.2: IoT Sensor → ESP32 → Backend → AI → Report (proposed architecture) 6
Figure 5.1: Crop Recommendation Methodology Flowchart 12
Figure 6.1: KrushiAI Application Dashboard..... 13
Figure 6.2: Farmer Details and Farm Registration Interface 13
Figure 6.3: Manual Soil Parameter Entry Interface..... 14
Figure 6.4: Developer Sensor Simulator..... 15
Figure 6.5: Soil Parameter Analysis Results..... 15
Figure 6.6: AI-Based Crop Recommendation Results..... 17
Figure 6.7: English and English–Marathi PDF Report Generation Interface 17
Figure 6.8: Generated Bilingual PDF Report (Test ID KAI-2026-51804) 18
Figure 6.9: Saved Soil Test History 20

LIST OF TABLES

Table 2.1: Comparison of the traditional approach and KrushiAI 4
Table 3.1: Functional requirements of Module 1 5
Table 3.2: Implementation status of the KrushiAI components 7
Table 5.1: Input features of the Random Forest model 10
Table 6.1: Farmer and farm details recorded for Test ID KAI-2026-51804 14
Table 6.2: Soil values and system status for Test ID KAI-2026-51804 (Data Source: Developer Simulator) 14
Table 6.3: Weather context retrieved from Open-Meteo (API-level run, Test ID KAI-2026-94421) . 15
Table 6.4: Ranked crop recommendation with model scores for Test ID KAI-2026-51804 16
Table 6.5: Model evaluation summary 18
Table 6.6: Ranked crop recommendation for Test ID KAI-2026-94421 20
Table 6.7: Verification status of tested functions 21

CHAPTER 1 — INTRODUCTION

1.1 Background

Every cropping season begins with decisions on which crop to grow, how suitable the available soil is for it, and whether the expected weather will support its growth. These decisions are influenced by soil nutrient levels, moisture, temperature, humidity, rainfall, and the farmer's own experience. In practice, farmers often lack an integrated view of soil analysis, environmental information, and data-driven crop guidance at the moment the decision has to be made.

KrushiAI addresses this gap. It is an Artificial Intelligence (AI) and Internet of Things (IoT) driven agricultural decision-support platform that brings soil parameters, location, weather context, a machine learning model, and a local agricultural knowledge context into a single workflow. The name "Krushi" is derived from the Marathi word for agriculture, and the project is developed with the Kopargaon region of Maharashtra in view.

1.2 Agriculture and Digital Transformation

Low-cost embedded controllers, wireless connectivity, open web APIs, and machine learning libraries now make it practical to digitise parts of the farming process that were earlier handled only by observation or laboratory work. In the Industry 4.0 perspective, a farm can be treated as a cyber-physical system in which sensors observe the field, a gateway transmits the observations, and a backend analyses them for the decision-maker [9], [11].

Digital transformation does not replace agronomic knowledge. Its purpose is to make relevant data available earlier, in a structured and recordable form. KrushiAI supports, and does not substitute, the judgement of the farmer and the advice of agricultural experts.

1.3 Need for Smart Agriculture

Several practical factors motivate a smart agriculture approach to crop selection:

- Soil nutrient status varies between fields and seasons, so generic recommendations may not suit a specific plot, and laboratory results may not be available when a sowing decision is made.
- Weather, soil, and past decision records are usually held in separate places and are not combined into one reviewable view.
- Records of earlier soil tests are rarely kept in a structured form, which makes it hard to compare seasons or to discuss a decision with an advisor.

1.4 Problem Statement

Farmers need a simple and integrated way to obtain data-supported crop suggestions based on the measured condition of their soil and the environmental context of their location, together with a record that can be shared and discussed with advisors. Existing practice either relies on experience alone or distributes the required information across several unconnected sources. A formal statement of the problem, with the identified research and technology gap, is presented in Chapter 2.

1.5 Objectives

The objectives of the project are:

1. To design an agricultural decision-support platform, KrushiAI, that integrates soil, location, weather, and machine learning components.
2. To accept soil data (nitrogen, phosphorus, potassium, soil moisture, air temperature, and humidity) through manual entry, a clearly labelled developer sensor simulator, and a planned Internet of Things (IoT) sensor path.
3. To develop a Random Forest based crop recommendation model that outputs a ranked list of candidate crops with model scores.
4. To retrieve weather context for a given latitude and longitude using Open-Meteo, and to attach a local agricultural knowledge context to the explanation of recommendations.
5. To generate English and bilingual English–Marathi PDF soil testing and crop recommendation reports for every soil test.
6. To expose the functionality through a modular FastAPI backend and a web interface, to document the prototype honestly including its limitations, and to define a realistic path towards the full platform.

1.6 Scope of the Project

The scope of the work reported here is the AI-Based Crop Recommendation System, covering farmer registration, soil data input, soil parameter analysis, location and weather context, machine learning based crop recommendation, agricultural knowledge context, saved test history, and PDF report generation, together with the sensor architecture designed for IoT-based data acquisition.

Market price prediction, storage and post-harvest monitoring, profit optimisation, long-term weather forecasting, a mobile application, and a full agricultural knowledge base are outside the current scope and are discussed only as future work. KrushiAI is a decision-support prototype and does not provide guaranteed agricultural advice.

1.7 Project Domain

The project lies at the intersection of Artificial Intelligence, Machine Learning, the Internet of Things, Smart Agriculture, Data Science, and Decision Support Systems. It is a final-year project with an Industry 4.0 theme that combines embedded hardware (ESP32 and sensors) with backend software and a machine learning component.

1.8 Overview of KrushiAI

Figure 1.1 gives a one-page overview of the platform: soil and environmental data enter the system, are analysed by the backend and the Random Forest model, and are returned as a ranked recommendation and a PDF report.

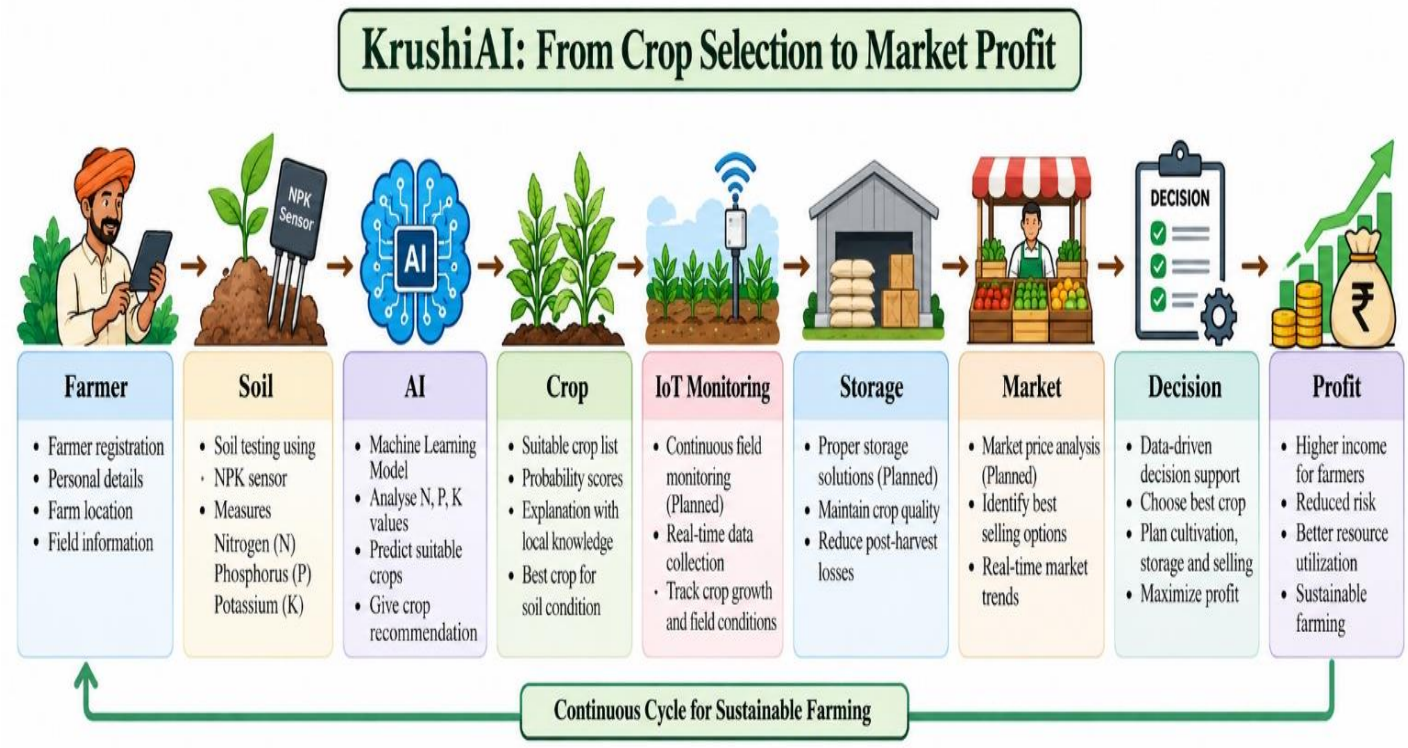


Figure 1.1: Overview of the KrushiAI platform

CHAPTER 2 — EXISTING SYSTEM AND PROBLEM STATEMENT

2.1 Existing Agricultural Decision-Making Methods

Farm-level decisions are traditionally made through inherited practice, observation of the field, discussion with neighbouring farmers, advice from input suppliers or extension workers, and, less frequently, laboratory soil test reports. These methods carry valuable local knowledge, but they are difficult to record, compare across seasons, or scale.

2.2 Traditional Soil Testing

In the conventional procedure, soil samples are collected, sent to a laboratory, analysed, and returned as a report, sometimes with general nutrient management advice. This remains the reference method and the standard against which any sensor-based approach must be validated. Its limitations are that sampling and analysis take time, the laboratory may be far from the farm, the test is not repeated frequently, and the report is not linked to current weather or to a ranked crop suggestion.

2.3 Manual Crop Selection

Manual crop selection depends on how well the farmer can relate soil condition, season, and expected weather to the requirements of each crop. It works well for familiar crops but is less reliable when soil conditions change, a new crop is considered, or the farmer is new to the region, and it produces no documented rationale.

2.4 Limitations of Existing Approaches

- Soil, weather, and location information are held in separate places and are not combined.
- Crop choice depends heavily on individual experience and is difficult to standardise.
- Results are rarely stored in a structured form, so past decisions cannot be reviewed.
- Reports are prepared manually and may not be available immediately.
- Sensor-based monitoring, where used, is often limited to a single parameter and is not connected to a recommendation engine.

2.5 Existing Digital Agriculture Solutions

Digital tools for agriculture can broadly be grouped into weather services, general advisory portals and mobile applications, stand-alone soil or environmental sensors, and academic crop recommendation models [9]–[12]. Weather services do not relate atmospheric data to the soil of a particular farm; advisory applications may not use measured soil values; stand-alone sensors leave interpretation to the user; and recommendation models are often presented in isolation, without data acquisition, location handling, or a deliverable report.

2.6 Research and Technology Gap

The gap addressed by KrushiAI is not the absence of any single technology, but the absence of an integrated and extensible pipeline that connects a soil data source (manual or sensor), location and weather context, a machine learning recommendation, an explanatory knowledge context, and a shareable report within one modular architecture that can grow towards storage and market decision support.

2.7 Problem Statement

To design and develop an agricultural decision-support platform that accepts soil nutrient, moisture, temperature, and humidity data from manual entry, a developer simulator, or an ESP32-based sensor setup, combines it with location and weather context, applies a machine learning model to rank suitable crops with model scores, attaches agricultural knowledge context, and produces a structured PDF report, while being clearly positioned as a decision-support prototype that requires validation by local agricultural experts and laboratory soil testing.

2.8 Need for the Proposed System

An integrated system reduces the effort of combining information, creates a stored record of every soil test, and gives the farmer or advisor a consistent starting point for discussion. Its modular architecture can later support monitoring, storage, and market modules. Table 2.1 compares the traditional approach with KrushiAI.

Table 2.1: Comparison of the traditional approach and KrushiAI

Parameter	**Traditional Approach**	**KrushiAI**
Soil analysis	Manual	Digital, AI-assisted
Crop selection	Experience, manual	ML-assisted
Weather integration	Separate	Integrated
Location	Manual	Latitude and longitude
IoT	Limited	ESP32-based architecture (planned)
Reporting	Manual	Automated PDF (English and bilingual)
Test records	Rarely stored	Saved test history
Decision support	Fragmented	Integrated

CHAPTER 3 — PROPOSED SYSTEM

3.1 Proposed System Overview

The proposed system, KrushiAI, is a layered software platform with an optional hardware front end. Soil data is typed in through the web interface, generated by a clearly labelled developer sensor simulator, or (in the planned design) measured by an ESP32-based sensor node. The FastAPI backend stores it, evaluates each soil parameter, associates the farm location, retrieves current weather context, and passes six features to a trained Random Forest classifier. The ranked output, with knowledge context and the soil and weather data, is compiled into a PDF report.

3.2 Functional Requirements

Table 3.1: Functional requirements of Module 1

ID	Requirement
FR1	The system shall register a farmer and allow the farmer record to be retrieved by identifier.
FR2	The system shall accept nitrogen (N), phosphorus (P), potassium (K), soil moisture, air temperature, and humidity through manual entry.
FR3	The system shall accept the same parameters from a sensor node or the developer simulator through a dedicated endpoint and return the latest stored reading.
FR4	The system shall analyse the soil values and report a status for each soil parameter.
FR5	The system shall accept latitude and longitude for a farmer and perform location analysis.
FR6	The system shall fetch weather context (temperature, humidity, wind speed, rainfall) for the given coordinates.
FR7	The system shall generate a ranked list of crop candidates with model scores.
FR8	The system shall attach local agricultural knowledge context to the recommendation.
FR9	The system shall generate and later retrieve a PDF report for each soil test, identified by a test ID.

3.3 Non-Functional Requirements

The system shall be modular, extensible, transparent (outputs labelled as model scores, with a decision-support disclaimer), reliable (invalid inputs rejected with an error), and usable by non-technical users through a readable web interface and report. Weather retrieval and sensor upload require internet access.

3.4 System Architecture

The architecture is organised into five logical layers, as illustrated in Figure 3.1.

- **Data acquisition layer:** manual entry through the web interface or API, the developer sensor simulator, and the planned ESP32 sensor node with the NPK, DHT11/DHT22, and soil moisture sensors.
- **Communication layer:** RS485/MAX485 between the NPK sensor and the ESP32, and an internet connection from the ESP32 to the backend; HTTP-based API calls to the Open-Meteo service.
- **Application layer:** the FastAPI service, served through Uvicorn, which exposes the endpoints and coordinates the modules.
- **Intelligence layer:** the Random Forest model and the local agricultural knowledge context.
- **Data and output layer:** the database layer that holds farmer and soil test records, the stored model artefact, and the generated PDF reports.

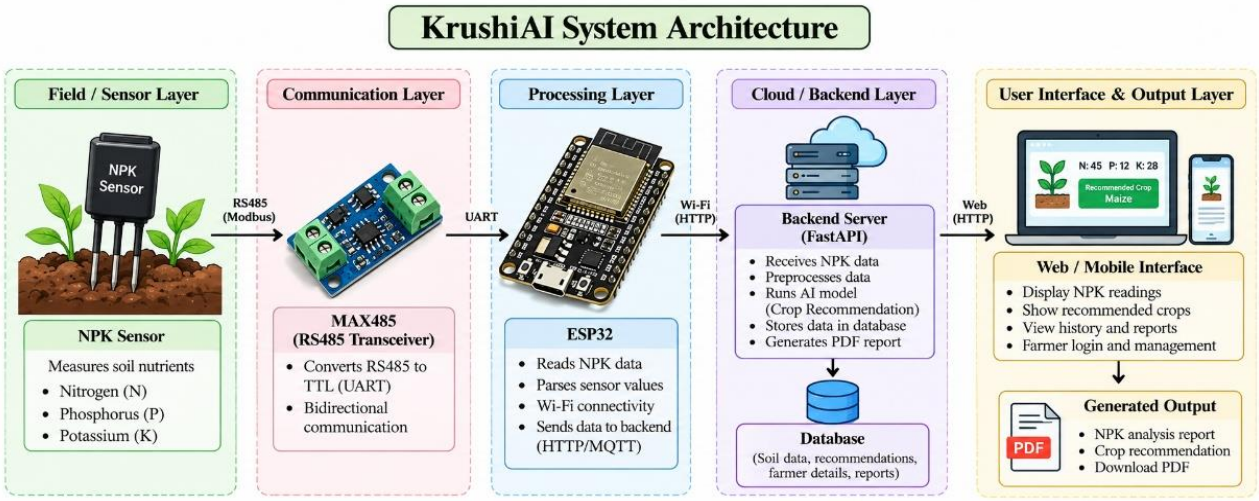


Figure 3.1: KrushiAI System Architecture

The data path of the planned sensor-based input, from the field sensors through the ESP32 to the backend, the recommendation engine, and the report, is shown separately in Figure 3.2. This path is a proposed architecture: it is described to define the intended design, and the physical sensor connection is not claimed as verified in this report.

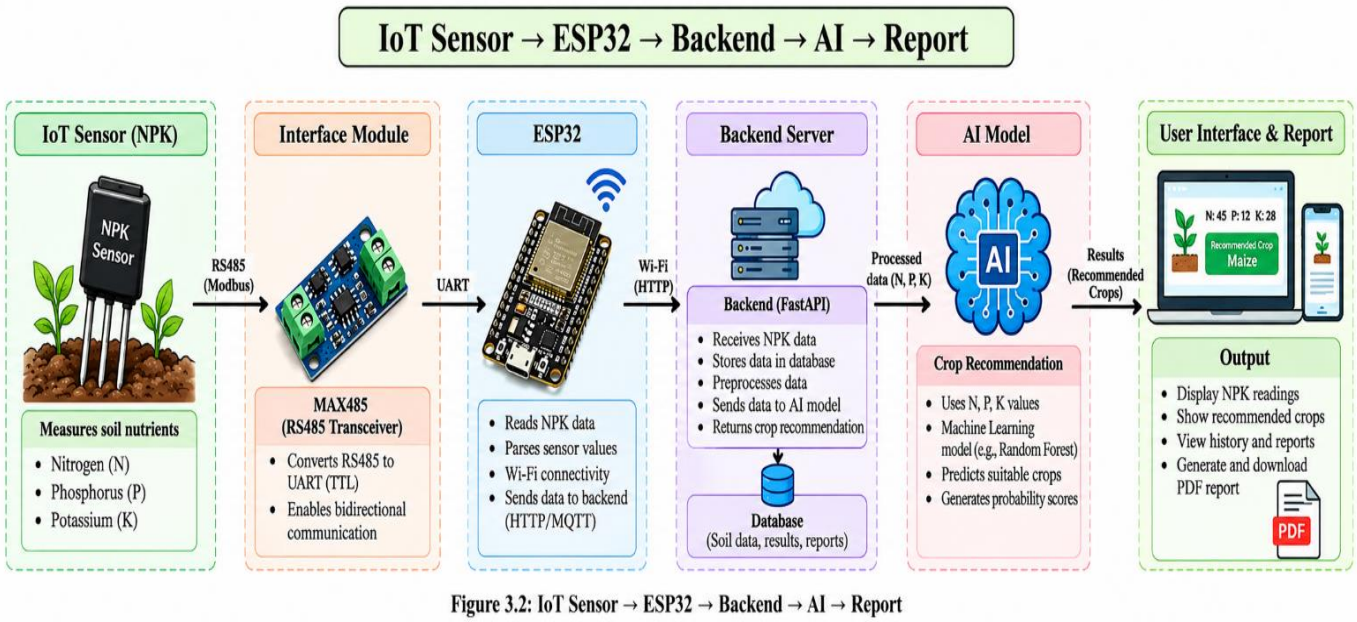


Figure 3.2: IoT Sensor → ESP32 → Backend → AI → Report

Figure 3.2: IoT Sensor → ESP32 → Backend → AI → Report (proposed architecture)

3.5 Module Description

The software modules and their endpoints are: farmer management (/api/farmers); soil data (/api/soil/manual, /read, /latest); soil analysis (/api/soil/analyze); location (/api/location/analyze); weather (/api/weather/fetch); crop recommendation (/api/crop/recommend), which also looks up the knowledge context; and report generation and retrieval (/api/soil-tests/{test_id}/report and /api/reports/{test_id}). Each module exchanges data with the others through the data layer.

3.6 Data Flow

Data enters as a soil record assigned to a soil test. The analysis, location, and weather modules attach parameter status, coordinates, and current conditions; the recommendation module then ranks the model scores and attaches knowledge context; and the report module compiles all stored results for the test ID into the PDF.

3.7 AI Recommendation Engine

The recommendation engine is a Random Forest classifier [1] built with Scikit-learn [2]. It takes N, P, K, temperature, humidity, and soil moisture and returns a model score for each crop class, which the backend sorts in descending order. A ranked list is deliberate, because the top candidates may differ only slightly. The model is loaded from a stored file, so it can be retrained without modifying the API.

3.8 IoT Integration

In the planned design, the ESP32 reads the NPK sensor through the MAX485 RS485 interface, the DHT11/DHT22, and the soil moisture sensor, and sends the combined reading to the backend over a network connection. The sensor input endpoint stores it as a soil record that follows the same pipeline as a manual entry, so the prototype can be tested through manual entry and the developer simulator while the sensor node is developed and calibrated.

3.9 Weather Integration

Weather context is obtained from Open-Meteo [5], [13] using the latitude and longitude supplied for the farmer. The retrieved values (temperature, humidity, wind speed, and rainfall) are stored with the soil test and shown in the report where available. The prototype does not perform long-term weather forecasting.

3.10 Report Generation

For each soil test, the report module compiles a PDF with the test ID, data source, farmer information, soil measurements and status, ranked crop recommendations with model scores, crop explanations, and a decision-support notice. The report can be produced in English and in a bilingual English–Marathi form. The test ID allows later retrieval.

3.11 Web Application Interface

The functionality of the backend is presented to the user through a web application built with React and Vite. The interface provides a dashboard, farmer and farm registration, soil input (manual entry and the developer sensor simulator), soil analysis results, crop recommendation results, report generation, and a saved test history.

3.12 Developer Sensor Simulator

The application includes a developer sensor simulator that generates a set of soil readings for demonstration and testing. Readings from the simulator are **simulated data, not live sensor readings**. The data source is recorded for each test, and the generated PDF carries a visible "SIMULATED DATA" notice stating that the readings were generated by a developer simulator and not by a live sensor.

3.13 Implementation Status

To avoid over-stating the prototype, Table 3.2 separates the components that are implemented and evidenced in this report from those that are simulated or still planned.

Table 3.2: Implementation status of the KrushiAI components

Component	Status	Evidence in this report
FastAPI backend and endpoints	Implemented and exercised	API-level test run (Section 6.10)
Manual soil entry and soil analysis	Implemented and exercised	Sections 6.2, 6.3
Random Forest crop recommendation	Implemented and exercised	Section 6.6; model score only
English and bilingual PDF report; developer simulator (simulated data only)	Implemented and exercised	Generated PDF for KAI-2026-51804
Weather retrieval (Open-Meteo)	Implemented; observed in API-level run	Section 6.4
Web interface and saved test history	Implemented; shown by screenshots	Sections 6.1 to 6.8
Planned ESP32 sensor hardware	Planned or under development	Not verified

CHAPTER 4 — COMPONENTS AND TECHNOLOGIES USED

This chapter describes the hardware and software components of KrushiAI and the role each plays in the architecture of Chapter 3. Exact sensor models, communication parameters, voltage ratings, and calibration values are not specified here; they are to be taken from the datasheets of the modules finally selected [4], [8]. The hardware components belong to the planned sensor node (Figure 3.2) and are not claimed as verified.

4.1 Hardware Components

4.1.1 ESP32

Purpose: The ESP32 is a low-cost microcontroller with built-in Wi-Fi and Bluetooth connectivity [4]. **Role in KrushiAI:** It is the planned IoT gateway and controller of the sensor node. It reads all sensors, forms a single reading, and transmits it to the backend. **Integration:** The ESP32 connects to the MAX485 module for RS485 communication, to the DHT11/DHT22 and soil moisture sensor through its input pins, and to the backend through the sensor input endpoint.

4.1.2 NPK Sensor

Purpose: The NPK sensor provides an estimate of the nitrogen, phosphorus, and potassium content of the soil in which it is placed. **Role in KrushiAI:** These three values are the main soil nutrient features of the recommendation model. **Integration:** It is connected to the ESP32 through the MAX485 module.

4.1.3 MAX485 / RS485

Purpose: RS485 is a differential serial standard suitable for robust communication over longer wires and in electrically noisy environments [8]. The MAX485 is a transceiver that converts between microcontroller logic-level serial signals and RS485 signals [14]. **Role in KrushiAI:** It forms the communication bridge between the ESP32 and the NPK sensor. **Integration:** The ESP32 serial pins connect to the transceiver, and the transceiver's bus terminals connect to the sensor.

4.1.4 DHT11/DHT22

Purpose: The DHT11 and DHT22 are digital sensors for air temperature and relative humidity. The DHT22 generally offers a wider range and better accuracy. **Role in KrushiAI:** Temperature and humidity are two of the six model features. **Integration:** The sensor is connected to a digital pin of the ESP32; the reading is merged with the NPK and moisture values before transmission.

4.1.5 Soil Moisture Sensor

Purpose: It measures the water content of the soil in the root zone around the probe. **Role in KrushiAI:** Soil moisture is the sixth model feature and is also reported in the PDF. **Integration:** The output is read by the ESP32 and converted to the moisture value expected by the backend.

4.1.6 Breadboard and Jumper Wires

Purpose: A breadboard and jumper wires allow the sensor node to be assembled and modified without soldering. **Role in KrushiAI:** They are used for prototype wiring, which makes testing of individual sensors and rearrangement of connections convenient. **Integration:** They link the ESP32, MAX485 module, and sensors.

4.2 Software Components

4.2.1 Python

Python is the common language for the backend, machine learning, and report code.

4.2.2 FastAPI

FastAPI [3] is a Python web framework for building APIs. It validates request data from declared types and generates interactive documentation (Swagger UI), which was used to test the endpoints during development.

4.2.3 Uvicorn

Uvicorn [15] is the ASGI server that runs the FastAPI application.

4.2.4 React and Vite

The web interface is built with React, a JavaScript library for component-based user interfaces, and Vite, a development server and build tool. The interface communicates with the FastAPI backend through its REST endpoints.

4.2.5 Pandas and NumPy

Pandas [6] loads and organises the tabular crop dataset during training and arranges the input features in the correct column order during inference. NumPy [7] provides the arrays used for feature data and model scores.

4.2.6 Scikit-learn

Scikit-learn [2] supplies the Random Forest implementation, the train/validation split, and the accuracy metric used in Section 6.9.

4.2.7 Random Forest

Random Forest [1] was selected because it handles numerical tabular features directly, needs little feature scaling, outputs class scores, and is a suitable baseline for small datasets (see Section 5.6).

4.2.8 Joblib

Joblib [16] is used to save the trained model to a file and load it at run time, so that the API does not need to retrain the model for every request.

4.2.9 Open-Meteo

Open-Meteo [5] is a weather data service with an HTTP API that returns weather values for a given latitude and longitude. It is used by the weather module to retrieve temperature, humidity, wind speed, and rainfall.

4.2.10 Database Layer

The database layer is accessed through SQLAlchemy, a Python object-relational mapping library. It stores farmer records, soil test records, location and weather context, recommendation results, and report references linking a test ID to its PDF, so that each module can read earlier results, the test history can be displayed, and a report can be retrieved later.

4.2.11 PDF Generation

A Python report-generation component compiles the stored results into a PDF with a fixed layout, saved against the test ID for retrieval through the report endpoint. The same component produces both the English report and the bilingual English–Marathi report.

CHAPTER 5 — METHODOLOGY AND IMPLEMENTATION

5.1 Development Methodology

The project followed an incremental, module-by-module approach: the data path was defined end to end, and each module was implemented and tested through its API endpoint before the next was integrated. Requirements (Table 3.1) were derived from Section 2.7; the main decisions were to accept manual, simulated, or sensor input, use six common features, present ranked model scores, keep weather as separate context, and produce a report for each test.

5.2 Requirement Analysis

The model is a replaceable component, so the pipeline could be validated early with a small demonstration dataset.

5.3 Data Collection

The crop recommendation model is trained on a small demonstration dataset that contains multiple crop classes, including rice, soybean, maize, groundnut, sugarcane, wheat, chickpea, cotton, onion, and tomato. Each record associates the six feature values with a crop label. The number of records, the number of records per class, and the original source of the dataset are not documented in this report; they should be confirmed from the training script before any quantitative claim about the data is made. The dataset is used to validate the software pipeline, and it should not be regarded as a representative regional agricultural dataset.

5.4 Data Preprocessing

The dataset is checked for completeness of the six feature columns and the crop label, and the columns are kept in a fixed order, which is reapplied at inference time to avoid a training and prediction mismatch. Random Forests are insensitive to monotonic feature scaling, so no normalisation is needed.

5.5 Feature Selection

The model uses the six features in Table 5.1. They were chosen because they can all be obtained from the planned sensor node, the developer simulator, or manual entry, which keeps the input paths equivalent.

Table 5.1: Input features of the Random Forest model

Feature	Meaning	Source
N	Nitrogen content of the soil	Manual entry / simulator / NPK sensor
P	Phosphorus content of the soil	Manual entry / simulator / NPK sensor
K	Potassium content of the soil	Manual entry / simulator / NPK sensor
Temperature	Air temperature (°C)	Manual entry / simulator / DHT11 or DHT22
Humidity	Relative humidity (%)	Manual entry / simulator / DHT11 or DHT22
Soil moisture	Water content of the soil	Manual entry / simulator / soil moisture sensor

5.6 Random Forest Model

A Random Forest [1] is an ensemble of decision trees. Each tree is trained on a randomly drawn sample of the training records, and at every split only a random subset of the features is considered. For classification, each tree gives a vote for a class and the forest combines these votes. When expressed as a score, the output is the fraction of trees, or the average of the tree-level class probabilities, that support each class. Averaging over many trees reduces the over-fitting of a single decision tree.

In KrushiAI this value is called the **model score**. It is not the probability of a successful harvest and does not guarantee yield or profit, because the model does not know about the local variety, irrigation, pests, market conditions, or management practices.

5.7 Model Training

The dataset is divided into a training portion and a held-out validation portion. The classifier is trained on the training portion, and its predictions on the validation portion are compared with the true labels to compute accuracy. The trained

model is saved to a file using Joblib. The split ratio, number of trees, random seed, and other hyperparameters are not reproduced in this report. On the demonstration dataset, the validation accuracy recorded by the team during development is approximately 40% (Section 6.9).

5.8 Model Prediction

At prediction time, the saved model is loaded and the six values are arranged in the training order. The model returns a score for each crop class, and the backend sorts them in descending order. The top candidates are returned together with their scores. In the reported tests, the five highest-ranked crops are shown.

5.9 Farmer Registration

A farmer record is created through the registration endpoint, which returns a farmer identifier that links later operations such as location analysis. The record includes the farmer's name, contact details, village, taluka, district, state, farm or plot name, and a preferred crop. A retrieval endpoint returns the stored record.

5.10 Manual Soil Data Entry

In manual mode, N, P, K, soil moisture, air temperature, and humidity are submitted to the manual soil endpoint, validated against declared types, and stored as a soil record with a soil test identifier.

5.11 Sensor and Simulator Data Acquisition

In sensor mode, the planned ESP32 firmware reads the NPK sensor over RS485, the DHT sensor, and the soil moisture sensor, and submits the combined reading to the sensor endpoint; the latest reading can be retrieved with the latest-soil endpoint. In the current prototype, the developer sensor simulator submits generated readings through the same path, and the data source of the test is recorded as "Developer Simulator". After storage, the record is processed like a manual entry.

5.12 Soil Analysis

The soil analysis endpoint evaluates the stored values against configured reference ranges and assigns a status to each parameter, which appears in the report. The individual threshold values are defined in the backend and are not tabulated here. The ranges should be reviewed by agricultural experts, and the report advises confirming measurement units and sensor calibration before soil-treatment decisions.

5.13 Location Analysis

The location endpoint accepts the farmer ID, latitude, and longitude and stores them with the soil test for weather retrieval. Administrative names are basic in the current prototype, since reverse geocoding is not implemented.

5.14 Weather Data Retrieval

The weather endpoint calls Open-Meteo for the coordinates and returns temperature, humidity, wind speed, and rainfall, which are stored with the soil test.

5.15 Crop Recommendation

The crop recommendation endpoint combines the stored soil record with the model. The recommendation process is as follows: (1) retrieve the soil record; (2) arrange N, P, K, temperature, humidity, and moisture in the training order; (3) obtain the score of each crop from the model; (4) rank the crops by score; (5) select the top candidates; (6) attach the agricultural knowledge context; and (7) store and return the result. The process is shown in Figure 5.1.

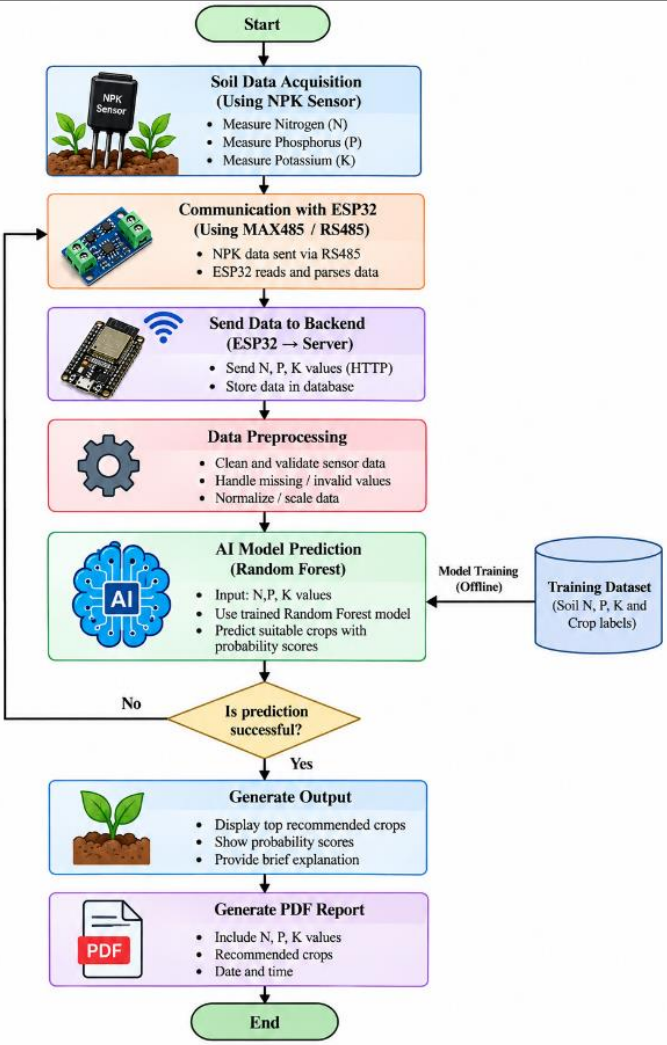


Figure 5.1: Crop Recommendation Methodology Flowchart

Figure 5.1: Crop Recommendation Methodology Flowchart

5.16 Agricultural Knowledge Integration

The recommendation output is supported by a local agricultural knowledge context. During the API-level test (Section 6.10), the explanation for the recommendation included a sugarcane entry retrieved from this local knowledge base. At present it is not a complete agricultural knowledge base, and no large language model is integrated.

5.17 PDF Report Generation

The report endpoint compiles all stored data for the soil test into a PDF assigned to the test ID, retrievable through the report retrieval endpoint. The report is available in English and in a bilingual form in which each heading, label, and explanation is given in English followed by Marathi. Its contents are listed in Section 3.10, and the generated example is discussed in Section 6.7.

5.18 API Implementation

All modules are exposed as REST endpoints in one FastAPI application with declared request and response structures, so incorrect inputs are rejected and interactive documentation is generated.

5.19 End-to-End Workflow

In an end-to-end run each step uses data stored by the previous steps, which allows the same soil test to be reproduced as a report at any later time using its test ID. In the web application, the user registers the farmer, enters or simulates the soil values, reviews the soil analysis, obtains the ranked crop recommendation, generates the English or bilingual report, and can later find the test in the saved test history.

CHAPTER 6 — RESULTS AND OUTPUT

This chapter presents the outputs of the KrushiAI web application for one representative soil test, Test ID **KAI-2026-51804**, whose data source is the Developer Simulator. All soil readings for this test are therefore **simulated data, not live sensor readings**. Only results obtained from the prototype are reported. Figure 6.1 shows the overall application dashboard from which the workflow starts.

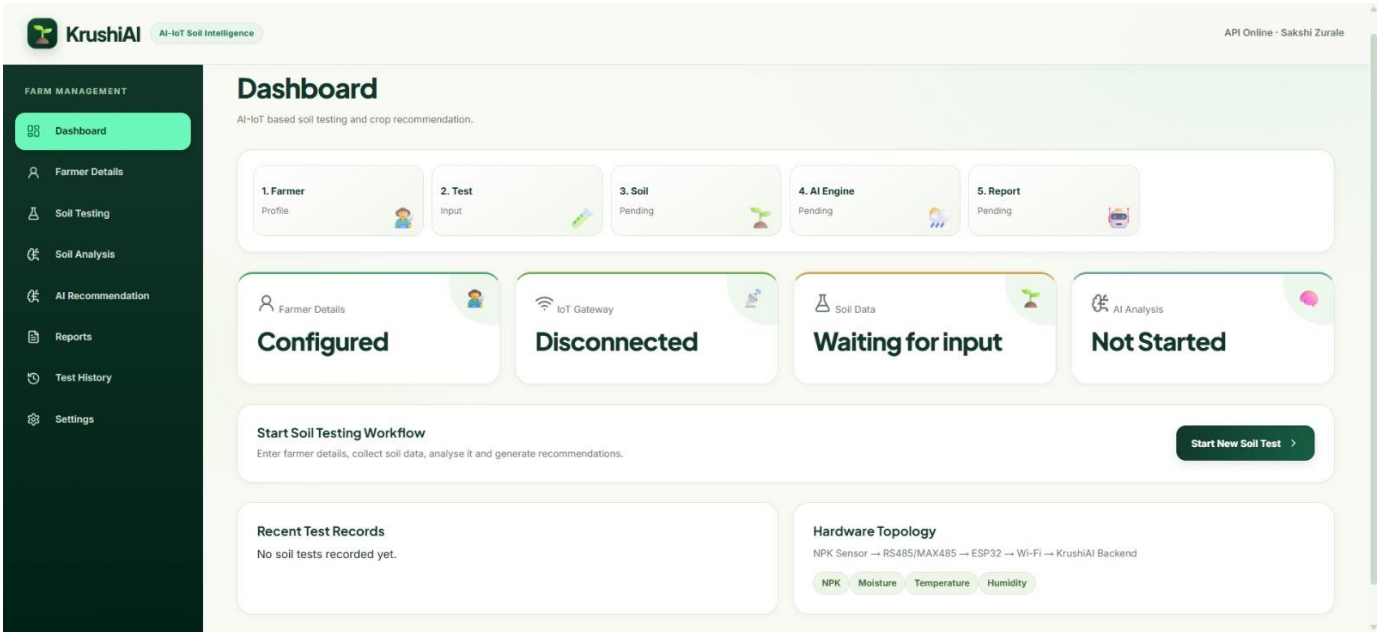


Figure 6.1: KrushiAI Application Dashboard

6.1 Farmer Registration Output

The farmer registration form creates a farmer record with the farm details and returns a farmer identifier, which is used in the subsequent steps of the test. Figure 6.2 shows the registration interface, and Table 6.1 lists the details recorded for this test.

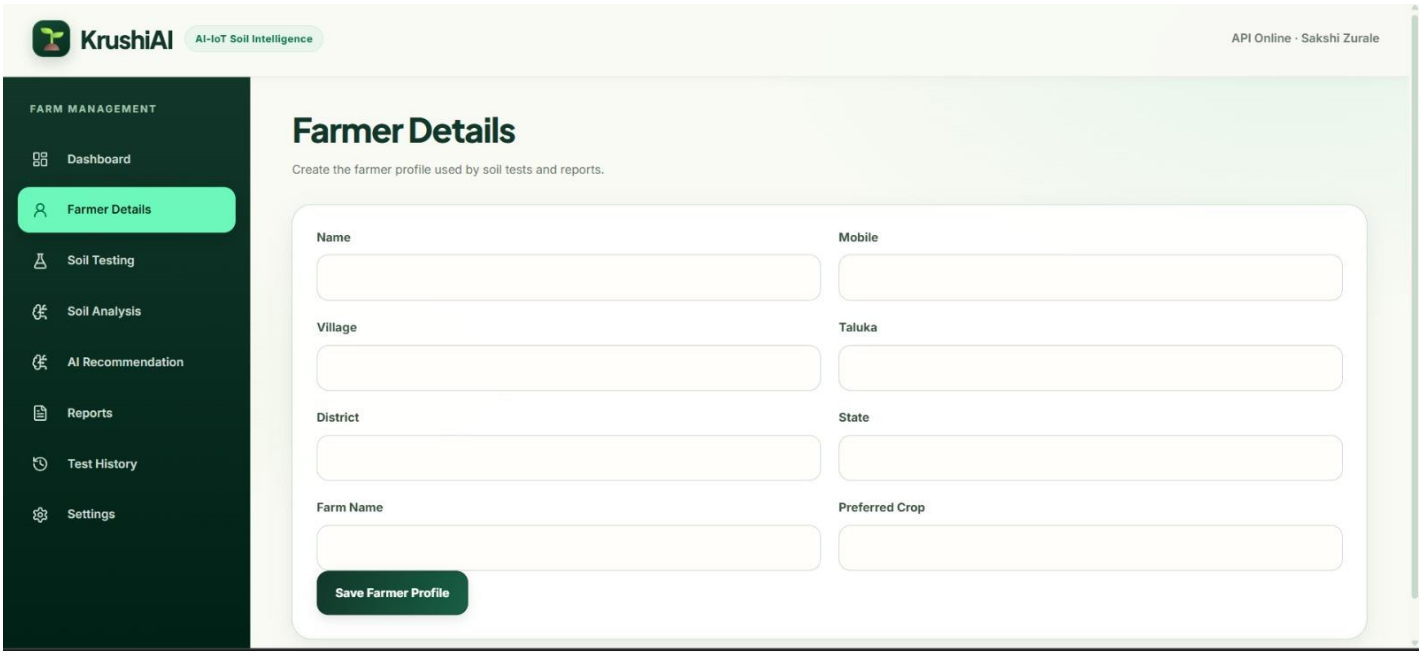


Figure 6.2: Farmer Details and Farm Registration Interface

Table 6.1: Farmer and farm details recorded for Test ID KAI-2026-51804

Field	Recorded value
Name	Sakshi
Village	Khairi
Taluka	Shrirampur
Farm / Plot	Zurale_Farm
Preferred crop	Maize

6.2 Soil Testing Output

Soil values can be provided by manual entry (Figure 6.3) or generated by the developer sensor simulator (Figure 6.4). The values stored for Test ID KAI-2026-51804, with their system status, are listed in Table 6.2; they were produced by the simulator and are not measurements from a physical sensor.

Table 6.2: Soil values and system status for Test ID KAI-2026-51804 (Data Source: Developer Simulator)

Parameter	Recorded value	System status
Nitrogen	55.0	Normal
Phosphorus	35.0	Normal
Potassium	45.0	Normal
Soil moisture	50.0	Normal
Air temperature	24.0	Normal
Humidity	68.0	Normal

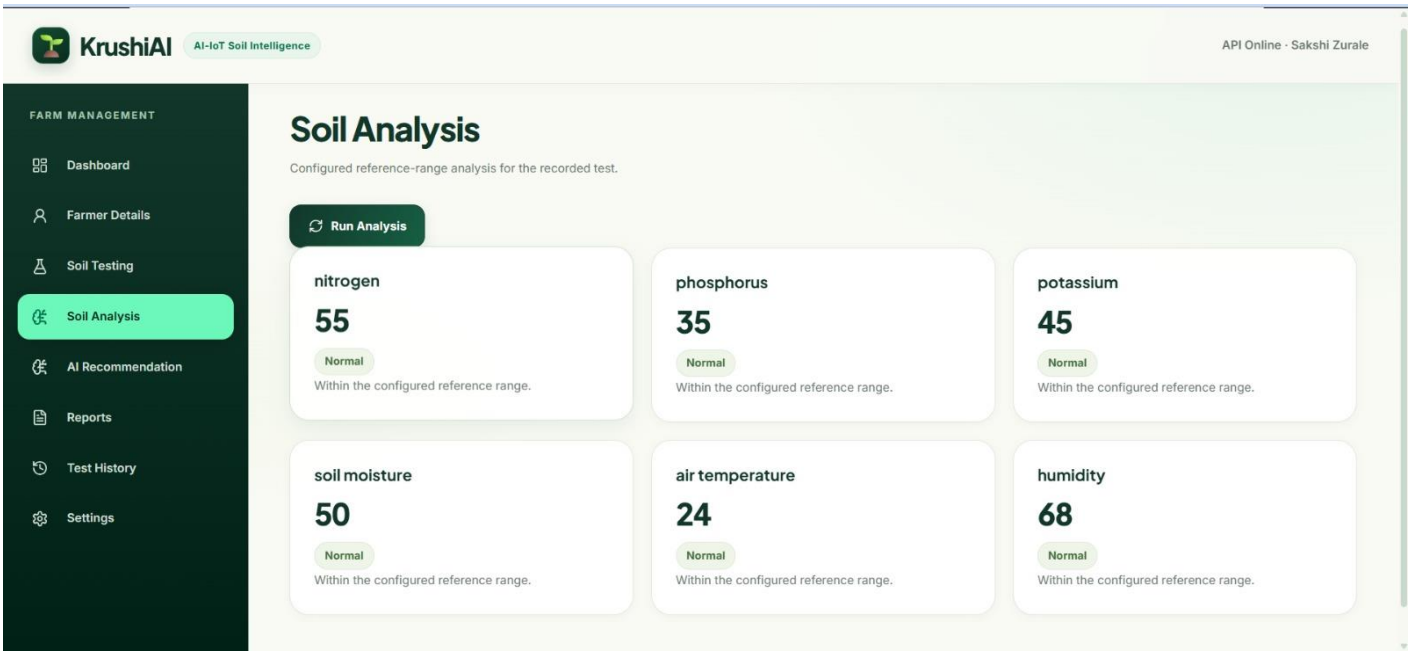


Figure 6.3: Manual Soil Parameter Entry Interface

Figure 6.4 shows the developer sensor simulator. The readings it produces are explicitly simulated; they exist to exercise the analysis, recommendation, and report pipeline while the physical sensor node is being developed.

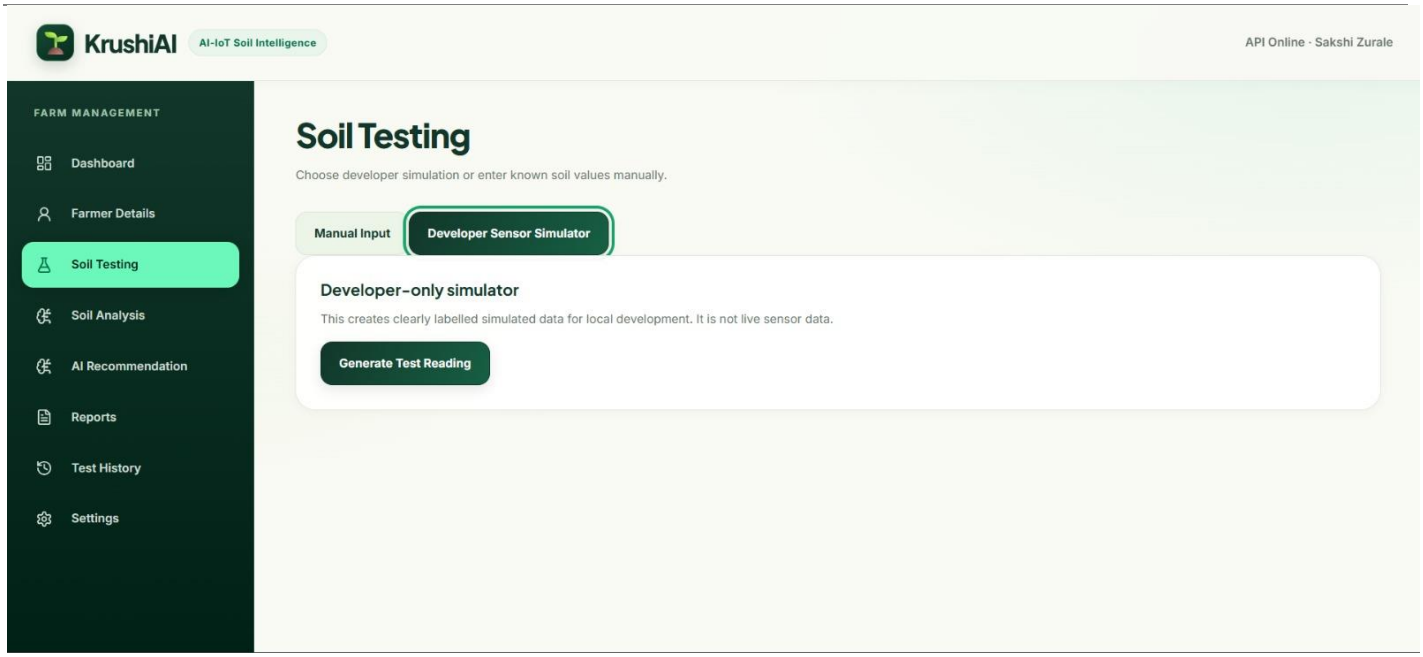


Figure 6.4: Developer Sensor Simulator

6.3 Soil Analysis Output

The soil analysis function evaluated the stored values and returned a system status for each parameter. As shown in Table 6.2 and Figure 6.5, all six parameters of this test were reported with the status Normal. The report reminds the reader to confirm measurement units and sensor calibration before making soil-treatment decisions.

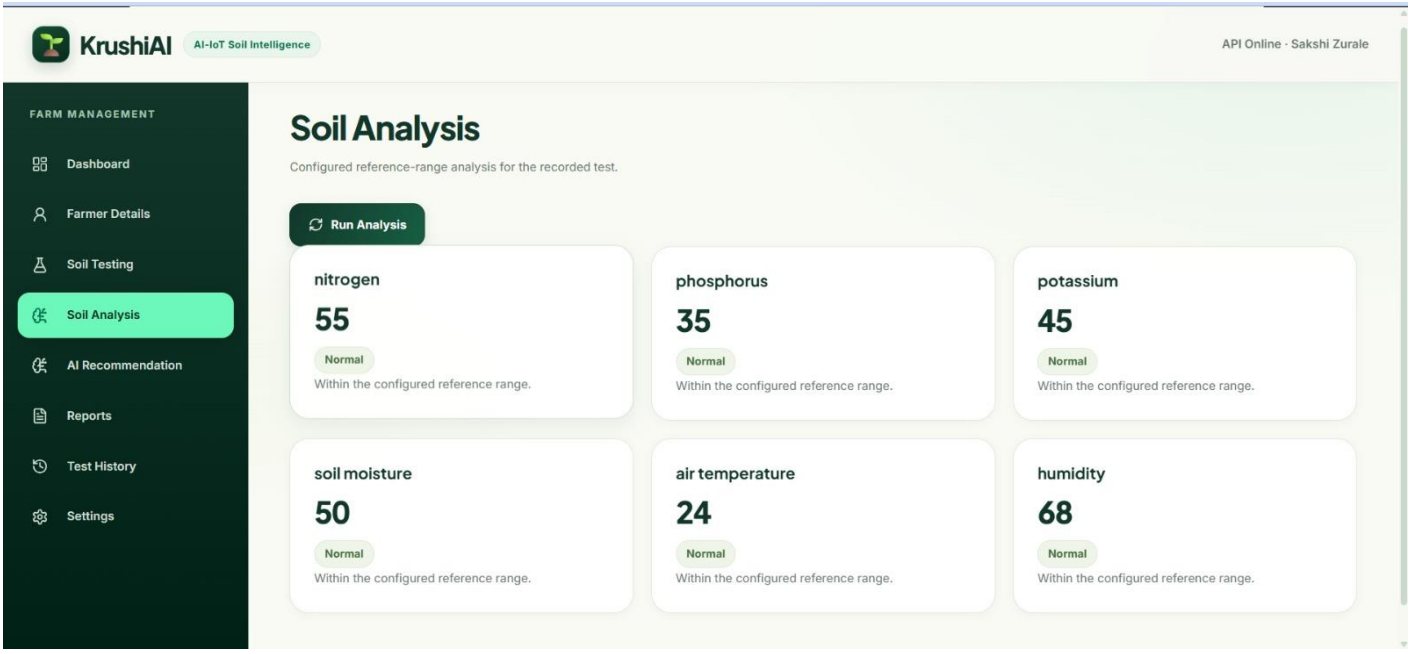


Figure 6.5: Soil Parameter Analysis Results

6.4 Weather Context Output

The weather module retrieves present conditions for the farm coordinates from Open-Meteo. The generated bilingual report for Test ID KAI-2026-51804 does not contain a weather block, so the weather values in Table 6.3 are taken from the earlier API-level test run (Test ID KAI-2026-94421, Section 6.10), in which the weather endpoint was exercised.

Table 6.3: Weather context retrieved from Open-Meteo (API-level run, Test ID KAI-2026-94421)

Parameter	Value
Temperature	31.9 °C
Humidity	37%
Wind speed	3.9 m/s
Rainfall	0 mm
Source	Open-Meteo

The weather temperature (31.9 °C) and humidity (37%) differ from the air temperature (25) and humidity (70) in the soil input of that run (see Section 6.10). This is expected: the weather values are conditions reported by the service at retrieval time, whereas the six model features are the soil-test values of Table 5.1; weather accompanies the recommendation as context only.

6.5 Agricultural Knowledge Context Output

In the API-level run, the explanation attached to the recommendation included a sugarcane entry retrieved from the local knowledge base. In the report for Test ID KAI-2026-51804, each crop explanation states that the model ranked the crop using the recorded inputs and that the score does not guarantee yield or profit.

6.6 Machine Learning Recommendation Output

For the soil values of Table 6.2, the model produced the ranked crop candidates of Table 6.4 and Figure 6.6.

Table 6.4: Ranked crop recommendation with model scores for Test ID KAI-2026-51804

Rank	Crop	Model score
1	Maize	33.2%
2	Onion	20.0%
3	Soybean	13.2%
4	Groundnut	10.0%
5	Tomato	10.0%

These values are model scores from the prototype and are not guaranteed crop success percentages. Maize is ranked first with a score of 33.2%, which is also the preferred crop recorded for the farm; this agreement should not be read as validation of the model. The lower-ranked scores are close together (groundnut and tomato are both 10.0%), consistent with a small demonstration dataset.

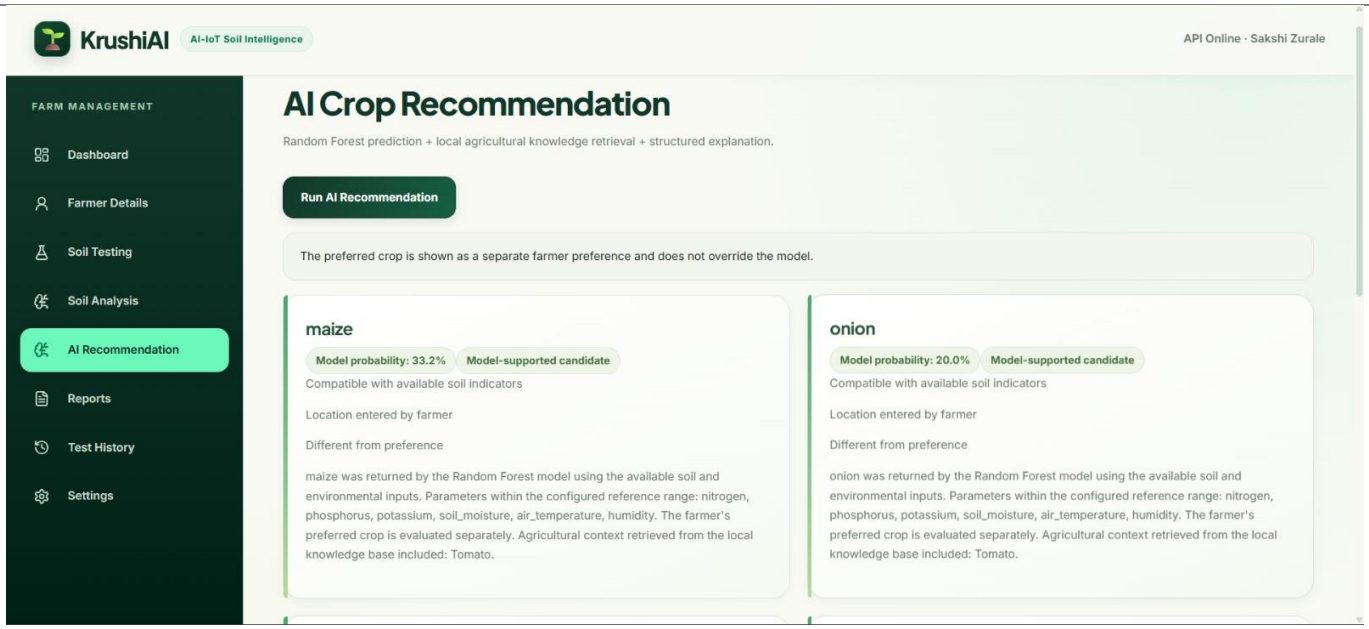


Figure 6.6: AI-Based Crop Recommendation Results

6.7 Report Generation Output

The report for Test ID KAI-2026-51804 was generated in the bilingual English–Marathi form (Figure 6.7). It contains the farmer information, soil measurements with status, ranked crops with model scores and suitability labels, and per-crop explanations; it carries the "SIMULATED DATA" notice and a notice that it is a decision-support aid and not a substitute for laboratory testing or expert advice. An extract is shown in Figure 6.8.

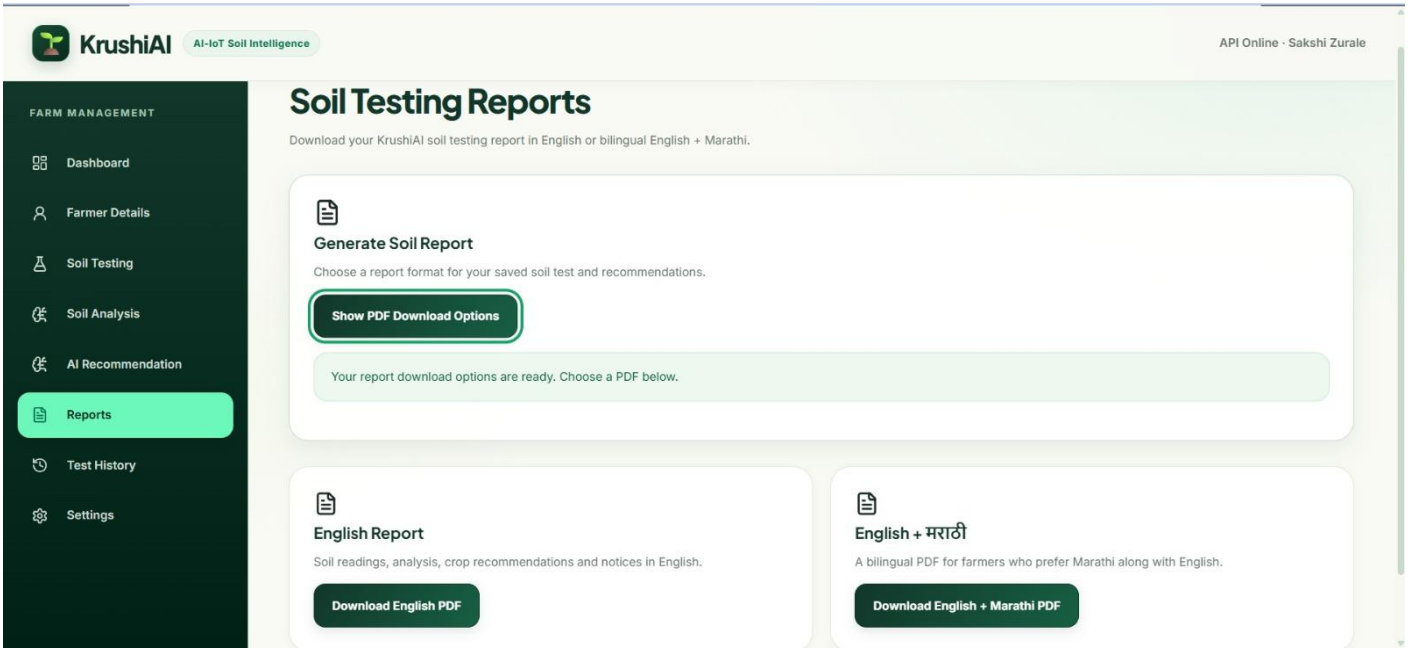


Figure 6.7: English and English–Marathi PDF Report Generation Interface

KrushiAI — Soil Testing & Crop Recommendation Report
मृदा परीक्षण आणि पीक शिफारस अहवाल

Soil analysis and AI-assisted crop recommendations
मातीचे विश्लेषण आणि कृत्रिम बुद्धिमत्तेवर आधारित पीक शिफारसी

Test ID: KAI-2026-51804
चाचणी क्रमांक: KAI-2026-51804

Data Source: Developer Simulator
माहितीचा स्रोत: Developer Simulator

SIMULATED DATA
प्रात्यक्षिकासाठी तयार केलेली माहिती
These readings were generated by a developer simulator, not by a live sensor.
ही माहिती डेव्हलपर सिम्युलेटरमधून तयार केली आहे; ती प्रत्यक्ष सेन्सरमधून घेतलेली नाही.

Farmer Information / शेतकऱ्याची माहिती

Name नाव	sakshi
Mobile मोबाईल क्रमांक	8408945372
Village गाव	Khairi
Taluka तालुका	Shrirampur
District जिल्हा	Ahilyanagr
State राज्य	Maharshatra
Farm/Plot शेताचे नाव	Zurale_Farm
Preferred Crop पसंतीचे पीक	Maize

Soil Measurements / मातीचे मोजमाप

Values are shown as recorded. Confirm measurement units and sensor calibration before making soil-treatment decisions.
मूल्ये प्रणालीमध्ये नोंदवल्याप्रमाणे दाखवली आहेत. मातीवर उपचार करण्यापूर्वी मोजमापाची एकके आणि सेन्सरचे कॅलिब्रेशन तपासा.

Parameter घटक	Recorded value नोंदवलेले मूल्य	System status प्रणालीतील स्थिती
Nitrogen नायट्रोजन	55.0	Normal सामान्य
Phosphorus स्फुरद	35.0	Normal सामान्य
Potassium पालाश	45.0	Normal सामान्य
Soil Moisture मातीतील ओलावा	50.0	Normal सामान्य
Air Temperature हवेचे तापमान	24.0	Normal सामान्य

+

Parameter घटक	Recorded value नोंदवलेले मूल्य	System status प्रणालीतील स्थिती
Humidity आर्द्रता	68.0	Normal सामान्य

AI Crop Recommendations / AI आधारित पीक शिफारसी

The model score is not the probability of a successful harvest and does not guarantee yield or profit.
मॉडेलचे गुण म्हणजे पीक यशस्वी होण्याची शक्यता नव्हे. त्यावरून उत्पादन किंवा नफा निश्चित होत नाही.

Crop पीक	Model score मॉडेल गुण	Suitability label योग्यतेचे वर्णन
maize मका	33.2%	Model-supported candidate मॉडेलने सुचवलेले पीक
onion कांदा	20.0%	Model-supported candidate मॉडेलने सुचवलेले पीक
soybean सोयाबीन	13.2%	Model-supported candidate मॉडेलने सुचवलेले पीक
groundnut भुईमूग	10.0%	Model-supported candidate मॉडेलने सुचवलेले पीक
tomato टोमॅटो	10.0%	Model-supported candidate मॉडेलने सुचवलेले पीक

maize / मका

Explanation: The model ranked maize using the recorded soil and environmental inputs. Its model score is 33.2%. This score is not the probability of a successful harvest and does not guarantee yield or profit.
स्पष्टीकरण: मका (maize) या पिकाला नोंदवलेल्या मातीच्या आणि पर्यावरणीय माहितीच्या आधारे मॉडेलने क्रम दिला आहे. मॉडेल गुण: 33.2%. हे गुण पीक यशस्वी होण्याची शक्यता दर्शवत नाहीत. त्यावरून उत्पादन किंवा नफा निश्चित होत नाही.

onion / कांदा

Explanation: The model ranked onion using the recorded soil and environmental inputs. Its model score is 20.0%. This score is not the probability of a successful harvest and does not guarantee yield or profit.
स्पष्टीकरण: कांदा (onion) या पिकाला नोंदवलेल्या मातीच्या आणि पर्यावरणीय माहितीच्या आधारे मॉडेलने क्रम दिला आहे. मॉडेल गुण: 20.0%. हे गुण पीक यशस्वी होण्याची शक्यता दर्शवत नाहीत. त्यावरून उत्पादन किंवा नफा निश्चित होत नाही.

soybean / सोयाबीन

Explanation: The model ranked soybean using the recorded soil and environmental inputs. Its model score is 13.2%. This score is not the probability of a successful harvest and does not guarantee yield or profit.
स्पष्टीकरण: सोयाबीन (soybean) या पिकाला नोंदवलेल्या मातीच्या आणि पर्यावरणीय माहितीच्या आधारे मॉडेलने क्रम दिला आहे. मॉडेल गुण: 13.2%. हे गुण पीक यशस्वी होण्याची शक्यता दर्शवत नाहीत. त्यावरून उत्पादन किंवा नफा निश्चित होत नाही.

groundnut / भुईमूग

Explanation: The model ranked groundnut using the recorded soil and environmental inputs. Its model score is 10.0%. This score is not the probability of a successful harvest and does not guarantee yield or profit.
स्पष्टीकरण: भुईमूग (groundnut) या पिकाला नोंदवलेल्या मातीच्या आणि पर्यावरणीय माहितीच्या आधारे मॉडेलने क्रम दिला आहे. मॉडेल गुण: 10.0%. हे गुण पीक यशस्वी होण्याची शक्यता दर्शवत नाहीत. त्यावरून उत्पादन किंवा नफा निश्चित होत नाही.

tomato / टोमॅटो

Explanation: The model ranked tomato using the recorded soil and environmental inputs. Its model score is 10.0%. This score is not the probability of a successful harvest and does not guarantee yield or profit.
स्पष्टीकरण: टोमॅटो (tomato) या पिकाला नोंदवलेल्या मातीच्या आणि पर्यावरणीय माहितीच्या आधारे मॉडेलने क्रम दिला आहे. मॉडेल गुण: 10.0%. हे गुण पीक यशस्वी होण्याची शक्यता दर्शवत नाहीत. त्यावरून उत्पादन किंवा नफा निश्चित होत नाही.

Important Notice / महत्त्वाची सूचना

This report is a decision-support aid, not a substitute for laboratory soil testing or advice from a qualified local agricultural expert. Confirm nutrient measurements, units, crop season, irrigation, and local conditions before acting on these recommendations.

Figure 6.8: Generated Bilingual PDF Report (Test ID KAI-2026-51804)

6.8 Saved Test History

Every completed soil test is stored with its test ID, so that it can be found again and its report retrieved. Figure 6.9 shows the saved soil test history of the application.

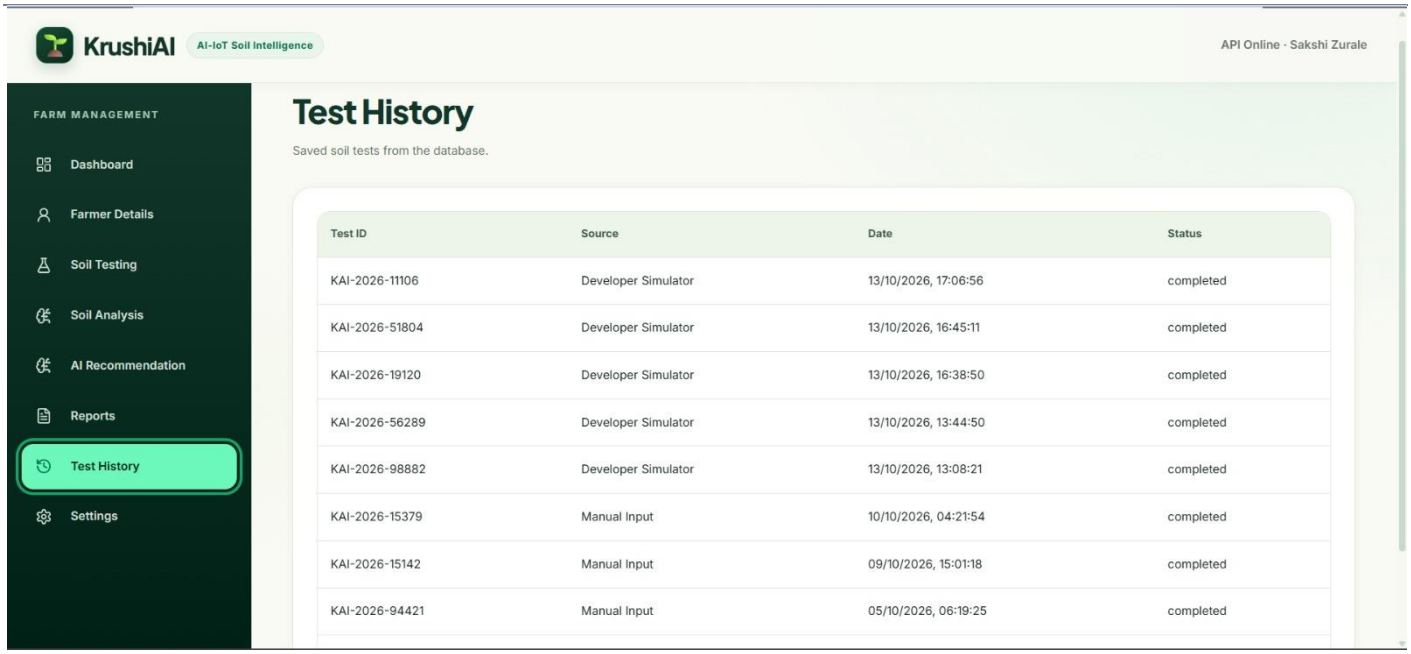


Figure 6.9: Saved Soil Test History

6.9 Model Evaluation

The Random Forest model obtained approximately **40% validation accuracy** on the demonstration dataset, as recorded by the team during development (Table 6.5). The evaluation output itself is not reproduced in this report, so the figure is reported as a team-recorded value.

Table 6.5: Model evaluation summary

Item	Result
Algorithm	Random Forest Classifier
Input features	N, P, K, temperature, humidity, soil moisture
Validation accuracy	Approximately 40% (team-recorded)
Dataset	Small demonstration dataset

This value is not production-level performance: the dataset was used to validate the software pipeline, and a larger, representative dataset and field validation are required before any reliability claim is made.

6.10 Earlier API-Level Test Run (Test ID KAI-2026-94421)

An earlier test of the complete pipeline was carried out through the API interface with Test ID KAI-2026-94421. It is retained because it also exercised weather retrieval and knowledge context. Its soil inputs were nitrogen 90, phosphorus 42, potassium 43, soil moisture 45, air temperature 25, and humidity 70, and its ranked output is given in Table 6.6. It is a different test from the one in Sections 6.1 to 6.8.

Table 6.6: Ranked crop recommendation for Test ID KAI-2026-94421

Rank	Crop	Model score
1	Rice	18.4%
2	Soybean	18.4%
3	Maize	18.0%
4	Groundnut	10.0%
5	Sugarcane	8.0%

In that run the top three candidates (18.4%, 18.4%, and 18.0%) were very close, so the model did not strongly separate rice, soybean, and maize for those inputs.

6.11 Verification Status of Tested Functions

Table 6.7 lists the evidence status of each function; Observed is used only where a prototype result is reported in this document.

Table 6.7: Verification status of tested functions

ID	Feature	Input / action	Expected result	Actual result	Status
T1	Farmer registration	Submit farmer and farm details	Farmer ID returned	Record created; ID used in later steps	Observed
T2	Manual soil input	Submit N, P, K, moisture, temperature, humidity	Soil record with test ID stored	Test ID returned; values stored	Observed
T3	Developer simulator	Generate simulated reading	Reading stored, marked as simulated	Test KAI-2026-51804; SIMULATED DATA notice in report	Observed
T4	Soil analysis	Analyse stored values	Status for each parameter	All parameters Normal (Table 6.2)	Observed
T5	Crop recommendation	Request ranking	Ranked crops with scores	Five ranked crops (Table 6.4)	Observed
T6	Weather retrieval	Request weather for coordinates	Weather values returned	Values returned in API-level run (Table 6.3)	Observed (API run)
T7	PDF report	Generate report for test ID	English / bilingual PDF	Bilingual PDF generated for KAI-2026-51804	Observed
T8	Test history retrieval	Open saved test history	Saved tests listed	Evidence: Figure 6.9	Pending screenshot
T9	Input validation	Submit missing or out-of-range values	Request rejected with error	No execution record supplied	Not verified
T10	Physical sensor path	ESP32 sends NPK, DHT, moisture reading	Reading stored	Hardware path not tested	Not verified

CHAPTER 7 — ADVANTAGES AND LIMITATIONS

7.1 Advantages

- **Data-driven crop recommendation:** crop candidates are ranked from soil and environmental values, rather than only from experience, and each candidate has a visible model score.
- **Integrated soil parameters:** nitrogen, phosphorus, potassium, moisture, temperature, and humidity are handled in a single record and a single analysis.
- **Weather context:** current conditions at the farm location can be shown with the recommendation.
- **Location context:** the use of latitude and longitude ties each test to a specific place and is the basis for weather retrieval.
- **IoT readiness:** the ESP32-based sensor architecture and the dedicated sensor input endpoint allow the manual or simulated path to be replaced by automatic acquisition.
- **Automated, bilingual reporting:** a structured PDF with a unique test ID is generated automatically, in English and in English–Marathi.
- **Saved test history:** each soil test is stored and can be found again through its test ID.
- **Modular architecture:** the model, weather source, or report layout can be changed with limited effect on other parts.
- **Reduced manual effort and expandability:** combining soil data, weather, and recommendation into one report removes several manual steps, and the same data model can support storage, market, and monitoring modules later.
- **Explainable recommendations:** ranked scores, explanations, and knowledge context are easier to discuss than a single label.

7.2 Limitations

The limitations of the current prototype are stated plainly so that its results are not over-interpreted.

- The machine learning dataset is small and was used as a demonstration dataset.
- The validation accuracy recorded by the team is only about 40%, which is not suitable for production decisions.
- Recommendation scores are model outputs, not guaranteed crop success rates; in the API-level test the top three crops differ by less than one percentage point.
- The soil readings shown in the results are simulated; the physical sensor path (ESP32, NPK sensor, RS485 interface) has not been verified end to end.
- Administrative location names are basic, because reverse geocoding is not yet included.
- Sensor readings will depend on calibration and soil contact; NPK readings in particular should be compared with laboratory results.
- Weather retrieval needs an internet connection, and the current weather component provides present conditions rather than long-term forecasts.
- The system is a prototype with a limited knowledge base and has not been field-tested in this report.
- Recommendations must be validated with local agricultural experts and laboratory soil testing where appropriate.
- Market prediction, storage monitoring, and profit modules are not part of the completed Module 1.

CHAPTER 8 — APPLICATIONS

The prototype is intended for decision support and education. The applications below describe where the platform could be used after further data collection and validation.

8.1 Smart Farming. Farmers can record soil conditions and obtain ranked candidate crops before sowing, alongside their own experience.

8.2 Precision Agriculture. Per-plot soil and location data is consistent with managing field variability [10] and supports plot-wise records.

8.3 Soil Testing Assistance. The soil analysis and PDF report present readings in a structured form, supplementing laboratory testing.

8.4 Crop Planning. Ranked candidates with knowledge context can inform seasonal planning when several crops appear similarly suitable.

8.5 Agricultural Advisory Systems. Advisors can use stored tests and reports as a consistent starting point for advice.

8.6 IoT-Based Agriculture. The ESP32 architecture is a low-cost starting point for field monitoring; the backend could later accept several nodes.

8.7 Farmer Decision Support. Ranked output, weather context, and the decision-support notice support, but do not replace, the farmer's own decision. The bilingual report makes the output readable for Marathi-speaking users.

8.8 Educational and Research Applications. The project is a teaching example in embedded systems, machine learning, and web services, and a platform for testing models and datasets.

8.9 Agricultural Extension Services. Extension workers and farmer organisations could use it for demonstration camps and digital soil-test records, subject to validation.

CHAPTER 9 — FUTURE SCOPE

None of the modules described in this chapter is implemented in the current prototype. They represent a realistic direction of development towards the long-term vision of KrushiAI: **Crop Selection → Cultivation → Monitoring → Storage → Market → Profit**.

9.1 Larger Agricultural Dataset. A larger, documented dataset representative of local soils, seasons, and crops, ideally validated by agronomists, is the most important improvement.

9.2 Improved ML Models. With more data, other algorithms can be compared with the Random Forest baseline using cross-validation and class-wise metrics.

9.3 Deep Learning. Deep learning could be explored once a large, varied dataset (possibly time-series sensor data or imagery) is available.

9.4 RAG-Based Agricultural Knowledge System. The local knowledge context could become a retrieval-augmented system grounded in verified agricultural documents, with sources shown.

9.5 Agentic AI for Agricultural Decision Support. An agent layer could coordinate soil, weather, and knowledge tools to answer farmer queries, with safeguards and expert review.

9.6 Advanced Weather Forecasting. Forecast inputs such as expected rainfall could be added to the weather context for sowing-time decisions.

9.7 Real-Time IoT Monitoring. Completing and calibrating the ESP32 sensor node, with continuous streaming, field enclosures, and alerts, would allow monitoring during cultivation.

9.8 Soil and Crop Storage Monitoring. Additional sensors could monitor storage conditions such as temperature and humidity after harvest, to reduce losses.

9.9 Market Price Prediction. Reliable market price data could be analysed to estimate price trends for candidate crops.

9.10 Profit Optimisation. Combining suitability, cost, and market information could support profit-oriented comparison with transparent assumptions.

9.11 Mobile Application. A mobile application would give farmers direct access to data entry, results, and reports in the field.

9.12 Multilingual Farmer Interface. The reports are already available in English and Marathi; extending the interface itself and adding Hindi and other languages would make the platform accessible to more farmers.

9.13 Voice-Based Agricultural Assistant. A voice interface could help users with limited literacy or limited comfort with text-based interfaces.

9.14 Reverse Geocoding. Reverse geocoding would replace the basic location names with village, district, and state information and link region-specific knowledge.

9.15 Cloud Deployment. Cloud hosting would let multiple users and devices access the system, with attention to security, privacy, and cost.

CONCLUSION

Crop selection is influenced by soil condition, weather, location, and experience, and the information required for this decision is usually scattered and not available in a structured form. This project addressed that problem by proposing KrushiAI, an AI-IoT driven agricultural decision-support platform, and by implementing its first module, the AI-Based Crop Recommendation System.

The prototype accepts soil nutrient, moisture, temperature, and humidity values through manual entry and a clearly labelled developer sensor simulator, and defines a sensor architecture based on the ESP32, NPK sensor, RS485/MAX485 interface, DHT11/DHT22, and soil moisture sensor. A FastAPI backend analyses the soil parameters, records the farm location as latitude and longitude, and retrieves weather context from Open-Meteo. A Random Forest classifier using six features produces a ranked list of candidate crops with model scores, a local knowledge context supports the explanation, and an English or bilingual English–Marathi PDF soil testing and crop recommendation report is generated for each soil test. A web interface presents the workflow and a saved test history. The pipeline was demonstrated for Test ID KAI-2026-51804 (simulated data), with maize, onion, soybean, groundnut, and tomato as the five top-ranked candidates, and for the earlier API-level Test ID KAI-2026-94421.

The achievements of the current prototype are therefore the working end-to-end software pipeline, the modular API, the web interface, the sensor-ready design, and the automated bilingual report. At the same time, the model reached only approximately 40% validation accuracy on a small demonstration dataset, the output scores are not guaranteed crop success rates, the physical sensor path is not yet verified, and the knowledge base and location handling are basic. These results demonstrate that the pipeline works but do not demonstrate production-level recommendation quality.

Future improvements include a larger and more representative dataset, improved models, a richer knowledge system, a completed and calibrated sensor node, real-time monitoring, storage and market modules, and cloud deployment, as discussed in Chapter 9. KrushiAI demonstrates the feasibility of an integrated intelligent agriculture decision-support platform. Further dataset expansion, field validation, and evaluation by agricultural experts are required before production deployment, and its recommendations should always be validated with local agricultural experts and laboratory soil testing.

REFERENCES

[1] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] FastAPI, "FastAPI framework, high performance, easy to learn, fast to code, ready for production." [Online]. Available: https://fastapi.tiangolo.com. [Accessed: Oct. 14, 2026].

[4] Espressif Systems, "Technical documents (ESP32 series)." [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents. [Accessed: Oct. 14, 2026].

[5] Open-Meteo, "Free weather API." [Online]. Available: https://open-meteo.com. [Accessed: Oct. 14, 2026].

[6] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[7] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[8] Telecommunications Industry Association, *Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems*, TIA/EIA-485-A, 1998.

[9] P. P. Ray, "Internet of things for smart agriculture: Technologies, practices and future direction," *Journal of Ambient Intelligence and Smart Environments*, vol. 9, no. 4, pp. 395–420, 2017, doi: 10.3233/AIS-170440.

[10] A. McBratney, B. Whelan, T. Ancev, and J. Bouma, "Future directions of precision agriculture," *Precision Agriculture*, vol. 6, no. 1, pp. 7–23, 2005, doi: 10.1007/s11119-005-0681-8.

[11] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, "Big data in smart farming – A review," *Agricultural Systems*, vol. 153, pp. 69–80, 2017, doi: 10.1016/j.agsy.2017.01.023.

[12] S. Pudumalar, E. Ramanujam, R. H. Rajashree, C. Kavya, T. Kiruthika, and J. Nisha, "Crop recommendation system for precision agriculture," in *Proc. 2016 8th Int. Conf. Advanced Computing (ICoAC)*, 2017, pp. 32–36, doi: 10.1109/ICoAC.2017.7951740.

[13] P. Zippenfenig, "Open-Meteo.com Weather API," Zenodo, ver. 0.2.69, Jul. 2023, doi: 10.5281/zenodo.8112599.

[14] Analog Devices, "MAX485: Low-power, slew-rate-limited RS-485/RS-422 transceivers." [Online]. Available: https://www.analog.com/en/products/max485.html. [Accessed: Oct. 14, 2026].

[15] Uvicorn, "Uvicorn: An ASGI web server for Python." [Online]. Available: https://www.uvicorn.org. [Accessed: Oct. 14, 2026].

[16] Joblib Developers, "Joblib: Running Python functions as pipeline jobs." [Online]. Available: https://joblib.readthedocs.io. [Accessed: Oct. 14, 2026].